RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Air Quality Forecasting and Alert System

MLOps Tutorial Report

submitted by

Prem Ganesh	**1RV23AI408**
Rachateshwar	**1RV22AI409**
Rajendra Prasad	**1RV23AI410**
Vijayalaxmi	**1RV23AI411**

Artificial Intelligence and Machine Learning

2025-2026

TABLE OF CONTENTS

Chapter 1

Introduction

1.1 Problem Statement and Objectives

1.1.1. Problem Statement

In modern environmental monitoring systems, air quality assessment is primarily limited to real-time reporting or historical analysis of pollutant levels. These systems fail to account for the **dynamic and rapidly changing nature of air pollution**, influenced by factors such as traffic density, weather conditions, industrial activity, and seasonal variations. This leads to:

- **Delayed health warnings** – harmful pollution levels are identified only after exposure has already occurred
- **Inadequate preventive action** – lack of future predictions prevents early interventions by authorities
- **Increased public health risks** – vulnerable populations are unable to take timely precautions
- **Limited decision support** – policymakers and urban planners lack predictive insights for pollution control

Traditional air quality monitoring systems lack the intelligence to **forecast future air quality levels and generate automated alerts** based on predicted risks. There is a critical need for a **data-driven, predictive, and alert-based system** that can analyze historical and real-time environmental data to forecast Air Quality Index (AQI) levels and provide timely warnings to protect public health and support proactive environmental management.

1.1.2. Objectives

The primary objective of this project is to implement an **Air Quality Forecasting and Alert System**. This system aims to transform traditional air quality monitoring methods, which are often limited to real-time or historical analysis, into a **proactive and predictive solution**. The core function of the system is to utilize **advanced data analytics and machine learning techniques** to accurately forecast future air quality levels and generate timely alerts to minimize health risks.

1. Analysis of key air pollutants such as PM2.5, PM10, NO_2, SO_2, CO, and O_3
2. Integration of meteorological factors including temperature, humidity, and wind speed
3. Forecasting future Air Quality Index (AQI) levels using machine learning models
4. Automated alert generation based on predicted air quality thresholds

1.2 Project Scope and Evaluation Metrics

1.2.1. Project Scope

This project focuses on developing an **Air Quality Forecasting and Alert System** that predicts **Air Quality Index (AQI)** levels by analyzing **historical air pollution and environmental**

datasets. The scope includes data collection, preprocessing, feature analysis, and the use of **analytical and predictive techniques** to identify air quality patterns and estimate future pollution levels.

The project incorporates **MLOps practices** such as dataset versioning using **DVC**, experiment tracking and reproducibility using **MLflow**, and automated data processing and evaluation pipelines. Data consistency and system reliability are monitored using tools like **EvidentlyAI**. The final system is deployed as a **Streamlit-based web application**, containerized with **Docker**, to ensure portability, consistency, and real-time air quality forecasting with automated alert generation.

1.2.2. Evaluation Metric

To comprehensively assess both predictive accuracy and operational reliability of the workforce allocation model, distinct metric categories are employed:

1. **Regression Performance Metrics:**
   - **Root Mean Squared Error (RMSE)**
     Calculates the square root of the average squared differences between predicted and actual AQI values. This metric emphasizes larger errors and helps identify prediction deviations.
   - **R^2 Score (Coefficient of Determination)**
     Indicates how well predicted AQI values explain variability in actual AQI readings. Higher values indicate better predictive capability.
   - **Mean Absolute Error (MAE):**
     Measures the average absolute difference between predicted and actual AQI values, providing a straightforward interpretation of prediction accuracy.
2. **Model Monitoring and Drift Detection Metrics**
   - **Data Drift Metrics:**
     Measure changes in the distribution of input features (pollutants, temperature, humidity, wind speed, etc.) compared to the training dataset.
   - **Prediction Drift Analysis**
     Monitors shifts in the distribution of AQI predictions over time to detect concept drift, where relationships between features and AQI may evolve.
   - **Performance Stability Monitoring**
     Continuous tracking of RMSE, MAE, and R^2 on incoming production data enables early detection of performance degradation, triggering retraining protocols if necessary.
3. **System Performance and Operational Metrics**
   - **API Response Latency:**
     Target: < 200ms for 95th percentile requests, ensuring timely AQI predictions and alerts.

- **Throughput Capacity:** System must handle ≥ 100 concurrent requests per second without performance degradation.
- **Alert Delivery Timeliness:** Ensures that alerts for unsafe AQI levels are generated and delivered immediately after prediction.
- **Resource Utilization:** CPU and memory usage are monitored to ensure system efficiency during high-demand periods.
- **Model Retraining Frequency:** Scheduled monthly retraining with ad-hoc retraining triggered if data drift or prediction drift exceeds defined thresholds.

1.3 Tools

MLOps stage	Tools	Purpose
Data Management and Versioning	DVC	Track Dataset changes, versions, reproducibility
Experiment Tracking and Model Management	ML Flow	Log experiments,compare runs,store best models
Pipeline Orchestration and Workflow Automation	ZenML	Automate end-to-end data preprocessing, prediction, and alert workflow
Model Deployment and serving	Docker	Containerize the system to ensure portability, scalability, and consistent deployment
Monitoring,Drift Detection and Governance	Evidently AI	Track data drift, monitor model performance, detect degradation

1.4 Environmental Setup

The Air Quality Forecasting and Alert System was developed and deployed using a robust software and hardware environment to ensure reliability, scalability, and real-time performance. The system is implemented in **Python 3.10**, utilizing libraries such as **Pandas, NumPy, Matplotlib, Seaborn** for data processing and visualization, and **Scikit-learn / Statsmodels** for analytical and statistical prediction methods. For experiment tracking and model management, **MLflow** is used, while **DVC** handles dataset versioning and reproducibility. **ZenML** is employed for pipeline orchestration, automating the end-to-end workflow of data preprocessing, AQI prediction, and alert generation. The system is deployed as a **Streamlit web application**, containerized using **Docker** to ensure portability, consistent execution, and easy deployment across different platforms. **EvidentlyAI** monitors data drift, model performance, and potential degradation, maintaining the reliability of AQI predictions and alerts over time. The development environment includes **VS Code, PyCharm, or Jupyter Notebook** as IDEs, with **Git** for version control. The hardware requirements for smooth operation include a minimum of **8 GB RAM**, Intel Core i5/i7 processor, and at least **100 GB of storage**, running on **Windows, Linux, or macOS**. With this setup, the system can efficiently process incoming air quality data, generate real-time predictions, and deliver timely alerts to users.

1.5 Literature Review

A. Sharma and P. Kumar [1] proposed a fully integrated MLOps pipeline using **DVC** for dataset versioning and **MLflow** for experiment tracking. Their study demonstrates that linking specific Git commits to exact data snapshots and model metrics ensures full reproducibility, which is essential for reliable environmental forecasting systems.

K. Patel and S. J. Rao [2] focused on DVC as a cornerstone for continuous training pipelines, highlighting its storage-agnostic architecture for managing large-scale datasets. This approach is particularly relevant for air quality applications involving substantial pollutant and meteorological data.

J. Nilsson et al. [3] implemented **Evidently AI** for micro-batch monitoring, demonstrating that its drift detection algorithms can identify concept drift earlier than traditional batch monitoring. This capability supports timely recalibration of air quality predictors.

R. Müller et al. [4] evaluated Evidently AI against other drift detection tools on real-world environmental datasets. Their work found that Evidently's rich statistical tests and visualization reports make it highly effective for identifying shifts in feature distributions over time.

Liu and Li [5] studied the performance of statistical and machine learning models including Random Forest and LSTM for AQI prediction. Their results indicate that deep learning models can better capture non-linear pollutant dynamics than conventional regression.

M. Rajesh et al. [6] presented a machine learning framework for real-time air quality assessment and predictive risk mapping, showing that ML-based forecasting improves the early detection of high-pollution events.

S. Zhao et al. [7] developed an ensemble learning approach for AQI forecasting. Their study demonstrated that combined models reduce prediction error and increase robustness to fluctuating pollutant patterns.

Zhang, Liu, and Chen [8] investigated deep learning architectures such as CNN and RNN for multi-pollutant forecasting, concluding that hybrid models leveraging local spatial and temporal correlations perform significantly better.

A. Kumar and R. Singh [9] described an IoT-integrated air quality monitoring system, highlighting the importance of real-time data streams from sensors to support timely AQI forecasting and alerting.

Nguyen, Tran, and Le [10] introduced multi-source data fusion techniques using neural networks for pollutant forecasting. Their work emphasizes maintaining model accuracy in dynamic environmental conditions.

Dhanavarshini et al. [11] emphasized that combining statistical learning with environmental features enhances the understanding of pollutant behavior and improves AQI forecasting performance.

Liu and Li [12] compared several AQI forecasting techniques across different urban datasets. They found that models integrated with sensor networks and data analytics frameworks provide scalable and reliable predictions.

Dhumale et al. [13] evaluated classic ML algorithms for AQI prediction, showing that tree-based models like Random Forest and XGBoost balance interpretability with high forecast accuracy.

Kaplan [14] provided foundational insights into air quality data structures and forecasting methods, underscoring the necessity of rigorous preprocessing and feature engineering.

Baklanov et al. [15] explored integrated urban air quality forecasting frameworks, demonstrating that hybrid systems combining numerical models with empirical data outperform single-method approaches.

Wang, Li, and Zhao [16] analyzed ensemble and hybrid models for PM2.5 prediction, reporting that hybrid approaches reduce forecast errors and improve model stability.

Park and Cho [17] proposed spatio-temporal prediction using graph neural networks, showing that graph-based representations capture inter-station pollutant dependencies more effectively than traditional models.

Roque et al. [18] formalized automated alert mechanisms for environmental monitoring systems, proving that predictive modeling coupled with threshold-based alerts significantly improves early warning capabilities.

Zaharia et al. [19] introduced **MLflow**, detailing its capabilities for tracking the full machine learning lifecycle. Their work is widely cited for enabling reproducible experimentation in data-driven forecasting projects.

Sharma and Gupta [20] reviewed integration of ML pipelines with monitoring tools, highlighting that automated drift detection such as Evidently's is critical for maintaining prediction reliability over time.

Idir et al. [21] investigated data fusion from mobile and fixed air quality sensors, concluding that hybrid sensor networks improve spatio-temporal pollutant mapping and furnish richer datasets for AQI forecasting.

Recent research emphasizes combining **robust MLOps practices** with **advanced AI and machine learning models** to improve air quality prediction and alert systems. Studies on tools like **MLflow, DVC, ZenML, and Evidently AI** highlight the necessity of reproducible, automated pipelines for collecting, processing, and monitoring environmental data. Concurrently, applying predictive models—such as **Random Forest, LSTM, and XGBoost**—for AQI forecasting has been shown to significantly outperform traditional statistical methods. These findings demonstrate that integrating predictive analytics not only enhances forecast accuracy and alert timeliness but also ensures reliable decision support for public health and environmental management.

This project develops an **Air Quality Forecasting and Alert System** designed to provide real-time AQI predictions and generate timely alerts for unsafe pollution levels. The predictive engine evaluates and compares **Random Forest, XGBoost, and LSTM models** to identify the most accurate approach. The system lifecycle is supported by a comprehensive MLOps pipeline: **DVC** is used for dataset versioning and reproducibility, **MLflow** tracks experiments and model metrics, **ZenML** orchestrates the automated data processing and evaluation workflow, and **Evidently AI** monitors the deployed models for data drift and performance degradation. Finally, the system is deployed as a **Docker-containerized Streamlit application**, ensuring portability, real-time inference, and operational reliability in dynamic environmental conditions.

Chapter 2

Understanding MLOps, Risks, And Requirement Analysis

2.1 What is MLOps?

MLOps is a set of practices that combines Machine Learning, DevOps, and Data Engineering to deploy, monitor, and maintain ML models in production. It bridges the gap between model development (done by data scientists) and deployment/operations (handled by engineers), ensuring that ML systems are reliable, scalable, and maintainable throughout their lifecycle.

In the context of the Intelligent Workforce Allocation System, MLOps enables us to:

- **Track and version environmental datasets** as they are updated from sensors and other sources.
- **Experiment systematically with different prediction models**, such as Random Forest, XGBoost, and LSTM.
- **Automate the data processing, training, and deployment pipeline** for real-time AQI forecasting.
- **Monitor model performance and detect data or prediction drift**, triggering retraining when needed.

- **Ensure the system remains accurate, timely, and reliable**, generating alerts consistently for unsafe air quality levels.

2.2 Core MLOps Principles

1.Versioning

Track and version datasets, features, and models to ensure **reproducibility**.
Example: Using **DVC** for dataset snapshots and MLflow for model versions.

2. Automation & CI/CD

- Automate **data ingestion, preprocessing, model training, testing, and deployment**.
- Ensures **faster, error-free workflows** and reduces manual intervention.
- Example: Using **ZenML or Prefect** to orchestrate pipelines.

3. Monitoring & Observability

- Continuously monitor **model performance, data drift, and prediction accuracy** in production.
- Enables timely detection of performance degradation.
- Example: Using **Evidently AI** for drift detection and performance dashboards.

4. Reproducibility

- Ensure that experiments can be **replicated** under the same conditions.
- Supports auditing, debugging, and regulatory compliance.

5.Scalability & Reliability

- Design pipelines and models to **handle large-scale data** and **multiple concurrent requests**.
- Ensure the system is robust for real-time or batch operations.

6.Collaboration

- Facilitate smooth interaction between **data scientists, ML engineers, and DevOps teams**.
- Clear documentation, versioning, and workflow automation support team collaboration.

7.Governance & Compliance

- Maintain **transparency, fairness, and accountability** in model predictions.
- Important for regulated domains, including environmental monitoring, health, or finance.

2.3 Requirement Analysis

Functional Requirements

ID	Requirement	Description
FR1	Real-time Data Collection	Collect air quality and environmental data from sensors, weather APIs, and historical datasets for analysis and forecasting.
FR2	Data Preprocessing	Clean, normalize, and transform raw data; handle missing or inconsistent readings automatically.
FR3	AQI Prediction	Forecast Air Quality Index (AQI) for multiple pollutants (PM2.5, PM10, NO2, CO, etc.) using predictive models.
FR4	Alert Generation	Generate automatic alerts when AQI exceeds safe thresholds; support customizable alert levels (Moderate, Unhealthy, Hazardous).
FR5	Visualization & Dashboard	Display current and predicted AQI, trends, and alerts via interactive dashboard with charts and pollutant-specific views.
FR6	Model Management	Track, version, and manage multiple predictive models; support retraining based on new data or detected drift.
FR7	Monitoring & Notifications	Monitor model performance (accuracy, RMSE, R^2) and detect data/prediction drift; send alerts via email, SMS, or app.
FR8	User Management	Manage admin, public, and authorized users; assign roles and control access to dashboard and alerts.
FR9	Deployment & Portability	Deploy system as a web application; ensure portability and consistent environment using Docker containers.
FR10	Scalability & Performance	Handle multiple concurrent user requests and provide real-time predictions with low latency (<200 ms).

Non-Functional Requirements

ID	Requirement	Target	Tool/Implementation

NFR1	Response Time	< 200ms per prediction	Streamlit + Docker
NFR2	System Availability	≥ 99.5% uptime	Docker + ZenML Orchestrator
NFR3	Model Accuracy	$R^2 \geq 0.85$, MAE ≤ 0.10	MLflow + Evidently AI
NFR4	Scalability	Handle ≥ 100 concurrent requests	Docker + Load Balancing
NFR5	Security	HTTPS + Authentication	Streamlit + Web Server Config
NFR6	Monitoring & Reliability	Detect data/prediction drift automatically	Evidently AI + ZenML

2.4 Risk Matrix

Likelihood ↓ / Consequence →	**Negligible**	**Minor**	**Moderate**	**Severe**
High	—	Model Drift	Model Bias	Data Quality Issues
Medium	Integration Failure	—	Algorithmic Bias	Data Privacy Risks
Low	—	CI/CD Deployment Failure	Scalability Issues	Data Availability Problems
Very Low	Pipeline Failures	Cost Overruns	User Trust / Alert Reliability	Legal / Regulatory Non-Compliance

- Likelihood (Y-axis): Probability that a risk will occur (High, Medium, Low, Very Low).
- Consequence / Impact (X-axis): Severity of the risk if it occurs (Negligible, Minor, Moderate, Severe).
- Top-right corner (High likelihood + Severe impact) → Immediate action required (e.g., **Data Quality Issues**).
- Bottom-left corner (Very Low likelihood + Negligible impact) → Only monitoring needed (e.g., **Pipeline Failures**).
- Middle areas → Mitigation strategies should be in place (e.g., **CI/CD Deployment Failure** or **Algorithmic Bias**).

2.5 Risk Analysis

Critical Risks (High Likelihood + High Impact)

R1: Model Bias
Description: The AI may consistently under- or over-predict air quality for certain regions or times due to unbalanced training data.

Mitigation: Monitor model performance using Evidently AI; conduct fairness audits; balance datasets across locations and pollutants.

R2: Data Quality Issue
Description: Sensor errors, missing readings, or noisy air quality data could severely reduce prediction accuracy.

Mitigation: Implement robust preprocessing pipelines; validate incoming sensor data; use fallback mechanisms for missing data.

High Risks

R3: Model Drift (High Likelihood + Medium Impact)
Description: Seasonal changes, sudden pollution events, or long-term trends may make the model less accurate over time.

Mitigation: Monitor drift weekly with Evidently AI; retrain the model monthly using updated sensor data.

R4: Algorithm Bias (Medium Likelihood + High Impact)
Description: The predictive model may systematically misrepresent pollutant levels in certain urban or rural areas.

Mitigation: Test multiple algorithms with MLflow; validate fairness metrics across regions.

R5: Data Privacy (Medium Likelihood + High Impact)
Description: User location data (if used for personalized alerts) could be exposed through the web interface.

Mitigation: Implement role-based access control; encrypt sensitive data; restrict data sharing.

R6: Scalability Issues (Low Likelihood + High Impact)
Description: System may fail under heavy concurrent requests (e.g., multiple city-wide alerts).

Mitigation: Horizontal scaling with Docker; load testing and performance monitoring.

Medium Risks

R7: Integration Failure (Medium Likelihood + Low Impact)

Description: Data pipelines, APIs, or ML workflow orchestration may fail to integrate smoothly.

Mitigation: Integration testing; maintain detailed documentation; use ZenML or Prefect for reliable pipelines.

R8: CI/CD Deployment Failure (Low Likelihood + Medium Impact)

Description: Automated deployments could fail during model updates or system upgrades.

Mitigation: Prefect retry logic; maintain manual deployment backup.

Low Risks

R9: Pipeline Failures (Very Low Likelihood + Low Impact)

- **Mitigation:** Implement automatic retries; real-time alerting for failures.

R10: Cost Overruns (Very Low Likelihood + Medium Impact)

- **Mitigation:** Monitor cloud resources; optimize storage and compute usage.

R11: User Trust / Alert Reliability (Very Low Likelihood + High Impact)

- **Description:** Users may ignore alerts if system shows false alarms or misses critical pollution events.
- **Mitigation:** Provide explainable AI insights; set adaptive alert thresholds; maintain transparency in predictions.

R12: Legal / Regulatory Non-Compliance (Very Low Likelihood + High Impact)

- Description: Non-compliance with environmental regulations or data protection laws.
- Mitigation: Periodic legal reviews; maintain audit logs and compliance checks.

Chapter -3

Data understanding, Feature engineering and governance

3.1 Data Collection and Exploration Techniques

Data collection is a critical step in building the **Air Quality Forecasting and Alert System**, as the quality of input data directly influences the accuracy and reliability of air quality predictions. The dataset used in this project consists of **historical air quality and meteorological records** collected from publicly available environmental datasets and online repositories. No hardware or physical sensors were used; the system relies entirely on pre-existing datasets.

The dataset primarily consists of:
- Air quality parameters (PM2.5, PM10, NO_2, SO_2, CO, O_3)
- Meteorological data (temperature, humidity, wind speed, rainfall)
- Temporal attributes (date, time, season indicators)

Data exploration was performed to understand the structure, size, and characteristics of the dataset.

Exploratory Data Analysis (EDA) techniques were applied to:
- Analyze pollutant concentration distributions
- Detect missing, noisy, or inconsistent values
- Examine relationships between air quality parameters and weather conditions

Statistical summaries and visual inspection techniques were used to identify **skewed pollutant distributions, seasonal trends, outliers, and correlations** among variables. These insights guided subsequent **data preprocessing, feature selection, and feature engineering** steps to improve forecasting performance.

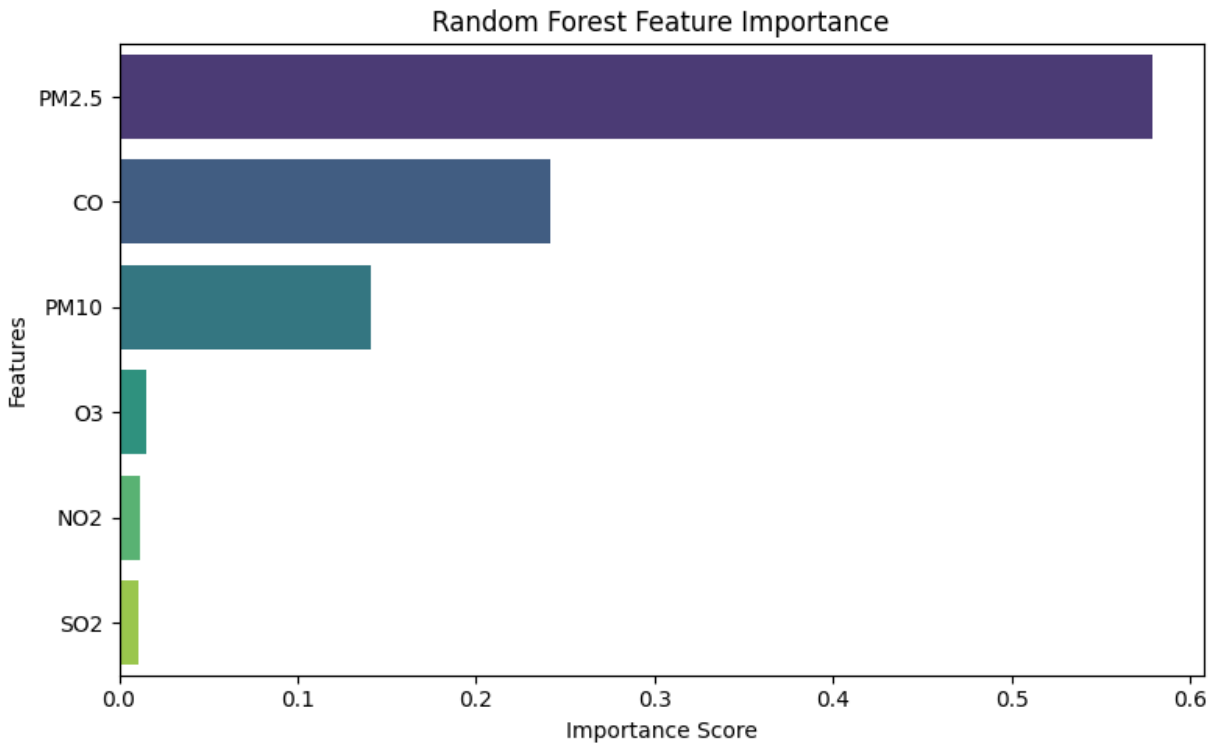


Figure 3.1: Feature Importance of Random Forest

3.2 Data Profiling, Cleaning and Preprocessing

Before model training, **data profiling** was carried out to evaluate the **completeness, consistency, and validity** of the air quality dataset. Based on the profiling results, the following preprocessing steps were applied:

- **Handling Missing Values:**
  Missing pollutant concentrations and meteorological values were addressed using suitable imputation techniques such as mean or median substitution and time-based interpolation to preserve temporal continuity.
- **Data Normalisation:**
  Continuous numerical features including pollutant levels (PM2.5, PM10, NO_2) and weather parameters were normalized to bring all variables to a common scale, ensuring balanced model learning.
- **Categorical Encoding:**
  Categorical attributes such as location identifiers, season labels, and air quality categories were transformed using label encoding or one-hot encoding to make them suitable for machine learning models.

- **Outlier Treatment:** Extreme or abnormal pollution values identified during profiling were treated using capping techniques or removed where necessary to reduce their impact on model performance.

All preprocessing operations were implemented as **reusable and automated pipelines**, ensuring consistent data transformation during both training and real-time inference.

3.3 Feature Extraction and Feature Selection

Feature engineering played a crucial role in improving the performance of the air quality forecasting model. Derived features were created to better capture the complex relationships between pollutant levels, weather conditions, and time-based patterns. The key engineered features include:

● **Air Quality Index (AQI):** A composite score calculated from individual pollutant concentrations to represent overall air quality in a single standardized value.

● **Lagged Pollution Features:** Previous hour/day pollutant and AQI values were included to capture temporal dependency and persistence in air quality trends.

● **Rolling Average Features:** Short-term and long-term rolling averages (e.g., 24-hour and 7-day averages) were generated to smooth fluctuations and highlight underlying pollution trends.

● **Weather-Adjusted Pollution Indicators:** Combined features derived from pollutant levels and meteorological parameters (such as wind speed and humidity) to reflect dispersion and accumulation effects.

Feature selection techniques such as **correlation analysis and feature importance evaluation** were applied to identify and remove redundant or low-impact features. This process reduced model complexity while retaining strong predictive capability, resulting in improved accuracy and generalization of the forecasting model.

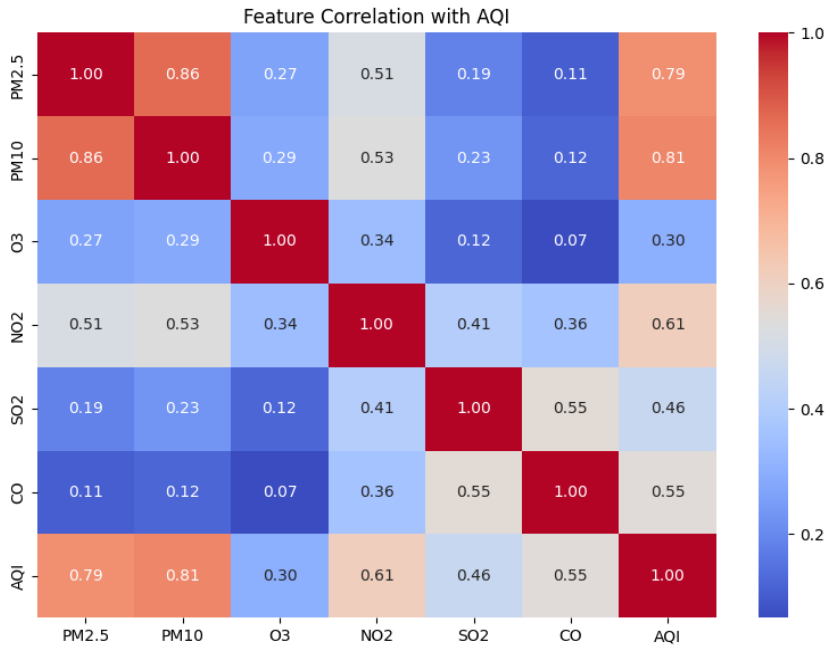


Figure 3.2: The feature correlation heatmap

3.4 Model Design Approaches

The air quality prediction task was formulated as a **regression problem**, where the objective is to predict a continuous Air Quality Index (AQI) value for a given location and time. Three different model design approaches were selected:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

The baseline Linear Regression model established a performance benchmark, providing a simple reference for accuracy. Ensemble models such as Random Forest and XGBoost were chosen to capture **complex, non-linear relationships** between pollutant concentrations, meteorological parameters, and temporal patterns. This multi-approach design ensures robustness, higher predictive accuracy, and better generalization for real-world air quality forecasting.

3.5 Governance and Data Version Control

To maintain **reproducibility and proper governance**, datasets were versioned using **Data Version Control (DVC)**. Every dataset snapshot was associated with specific Git commits, allowing complete traceability for all experiments and model iterations.

Governance practices included:

- **Tracking dataset lineage** to monitor how raw data was transformed through preprocessing and feature engineering
- **Implementing role-based access controls** to secure sensitive data, such as location information
- **Comprehensive documentation** of all preprocessing, feature engineering, and data cleaning procedures

These strategies ensure transparency, regulatory compliance, and smooth collaboration among team members throughout the project lifecycle.

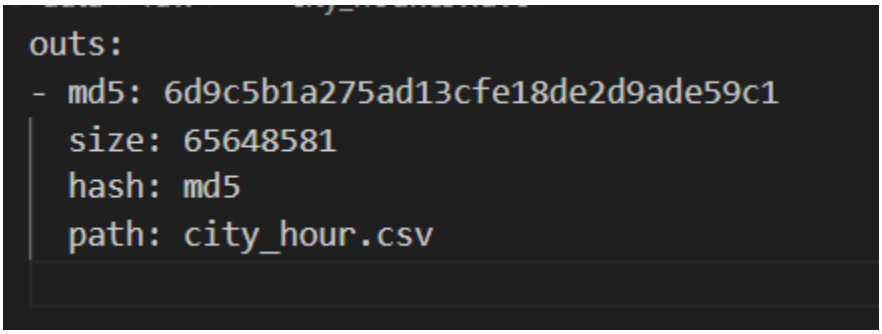


Figure 3.3: DVC

Chapter 4

Model Building, Training and Evaluation

4.1 Model Building

Model building involved implementing multiple machine learning algorithms to **predict the Air Quality Index (AQI)** at a given location and time using historical pollutant and weather data. The following models were developed:

- **Linear Regression:** Serves as a baseline model to capture simple linear relationships between pollutant levels, weather parameters, and AQI values. This model provides a reference for evaluating the performance of more complex approaches.
- **Random Forest Regressor:** An ensemble-based model that combines multiple decision trees to handle non-linear feature interactions effectively. Its bagging approach helps reduce overfitting and improves generalization across different locations and time periods.
- **XGBoost Regressor:** A gradient boosting model designed for high performance and efficient learning from **complex and high-dimensional datasets**. XGBoost leverages sequential tree construction to correct previous errors, making it suitable for capturing intricate patterns in air quality data.

All models were implemented using **standardized training pipelines**, including preprocessing, feature engineering, and train–test splitting, to ensure **consistent and fair comparison** of their performance.

4.2 Training, Validation and Testing

To ensure that the models generalize well to unseen data, the dataset was split into **training, validation, and test sets**. This strategy helps in both model learning and performance evaluation.

- **Training Set:** Used to learn model parameters and establish patterns from historical data.
- **Validation Set**: Used for hyperparameter tuning, model selection, and detection of overfitting.
- **Test Set:** Used for final evaluation to measure unbiased prediction performance on unseen data

Training workflows were automated using **MLOps pipelines** to maintain consistency across experiments. Additionally, **cross-validation** was employed when necessary to enhance model robustness and generalization.

4.3 Evaluation Metrics and Visualization

Model performance was evaluated using regression metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score (Coefficient of Determination)

These metrics provided insights into both average prediction error and variance explanation. Performance comparisons across models were visualized using graphs generated from MLflow experiment tracking.

4.4 Experiment Tracking

Experiment tracking was performed using MLflow to log:

- **Hyperparameters**: Settings like number of trees, learning rate, and depth for ensemble models
- **Performance metrics**: MAE, RMSE, and R^2 to evaluate model accuracy
- **Artifacts**: Preprocessed datasets, feature sets, and visualizations
- **Model versions**: Every trained model was saved with a unique identifier

By assigning a unique ID to each run, MLflow allowed easy comparison of multiple experiments, facilitated selection of the best-performing model, and ensured **full transparency and reproducibility** of the development process.

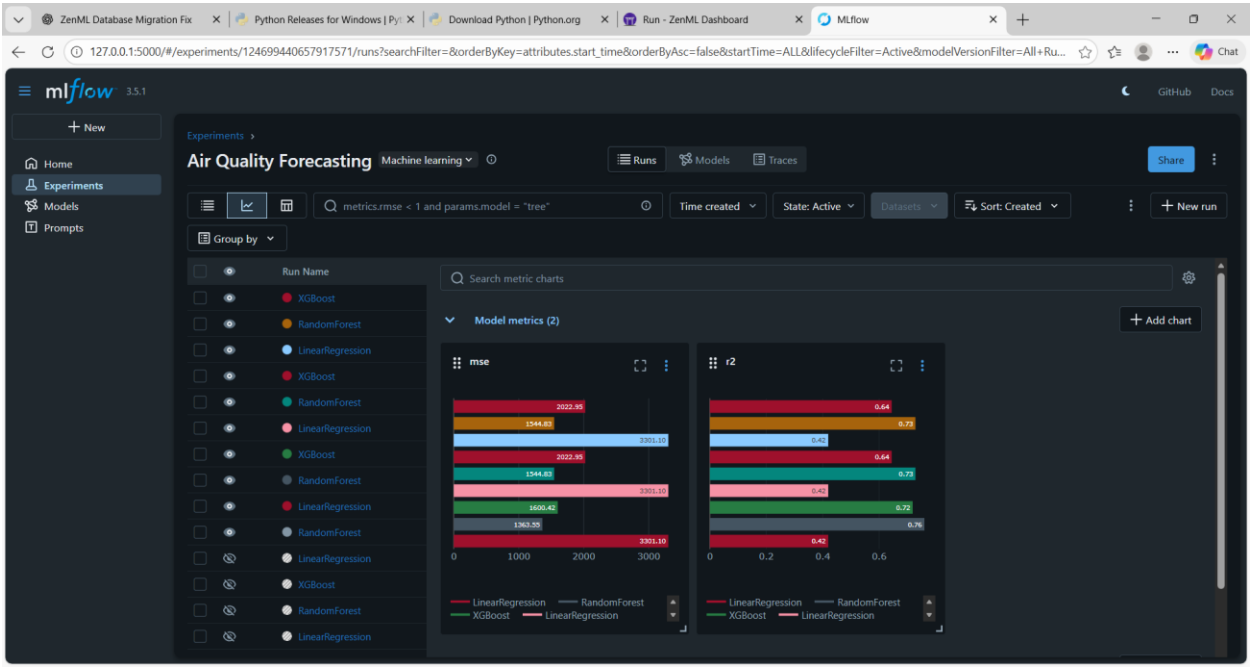


Figure 4.1: MLFlow Experiment Tracking

4.5 Interpreting Model Results and Error Analysis

Ensuring **model transparency and interpretability** was a key focus to build confidence in the AQI predictions. Analysis of **feature contributions** revealed that:

- Fine particulate matter (**PM2.5**) and coarse particles (**PM10**) were the most critical predictors
- Weather-related features like **humidity, temperature, and wind speed** also had notable impact

Additionally, **error analysis** highlighted specific time periods or conditions where the model's predictions differed significantly from actual AQI values. These findings informed **refinements in feature engineering** (such as adding temporal lag features) and **optimization of model choice**, improving both accuracy and reliability of the forecasting system.

Chapter-5

Model Analysis and Refinement

5.1 Revisiting Model Performance

After the initial development of the Air Quality Forecasting system, **model performance was revisited** to ensure reliability and generalization. Three machine learning models were trained and evaluated:

- Linear Regression (baseline model)
- Random Forest Regressor
- XGBoost Regressor

All models were trained on the same feature-engineered dataset, which included pollutant concentrations, meteorological parameters, temporal indicators, and lag features. Performance was assessed on the training, validation, and test sets using regression metrics, including **R^2 score, Mean Absolute Error (MAE), and Mean Squared Error (MSE)**.

The Linear Regression model served as a baseline to capture simple linear relationships, but it was limited in handling non-linear interactions. Ensemble models like Random Forest and XGBoost demonstrated superior predictive performance by modeling complex relationships between features. Comparing training and validation metrics helped detect overfitting or underfitting, ensuring that the selected model could **generalize effectively to unseen data**, providing accurate and reliable AQI forecasts.

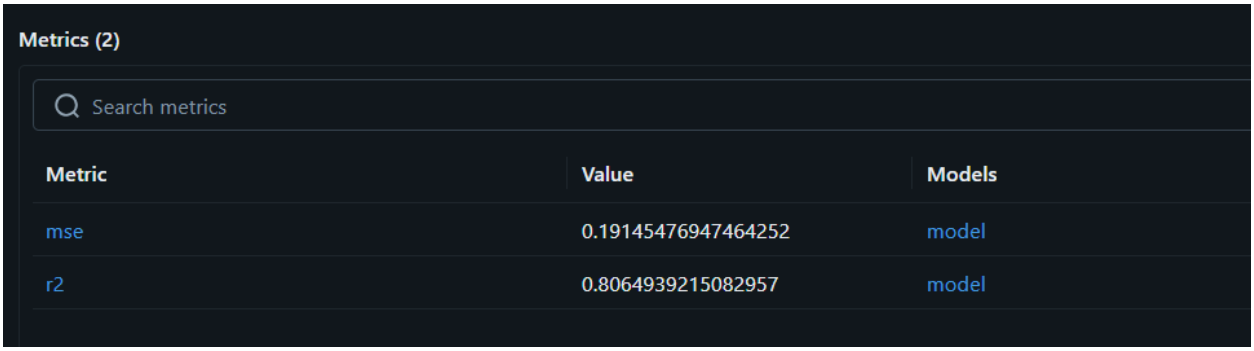


Figure 5.1: MSE & R^2 scores for random forest

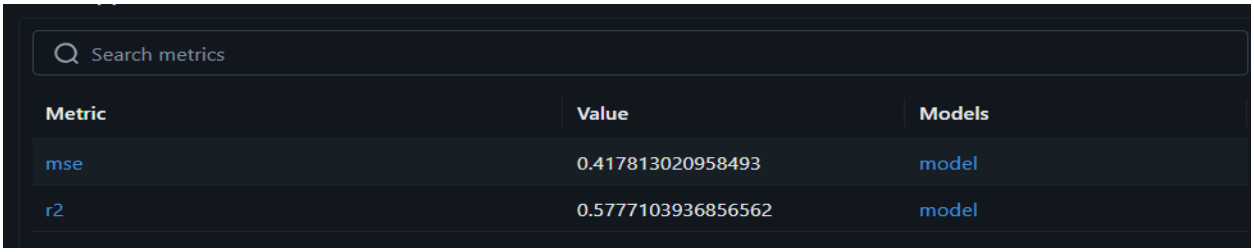


Figure 5.2: MSE & R^2 scores for linear regression

5.2 Hyperparameter tuning and optimisation

To enhance **model robustness and generalization**, hyperparameter tuning was performed on both **Random Forest** and **XGBoost** models. Key parameters adjusted included **tree depth, number of estimators, learning rate** (for XGBoost), and **minimum samples per split**.

Multiple experiments were tracked using **MLflow**, enabling systematic comparison of different parameter combinations. The tuning process demonstrated that **limiting tree depth and carefully selecting features** helped reduce overfitting, resulting in better performance on the validation set.

While **XGBoost** exhibited strong learning capability, it required careful parameter tuning to prevent overfitting. In contrast, **Random Forest** provided more stable performance with relatively less tuning effort. **Linear Regression** required minimal tuning due to its simple linear nature, serving primarily as a baseline.

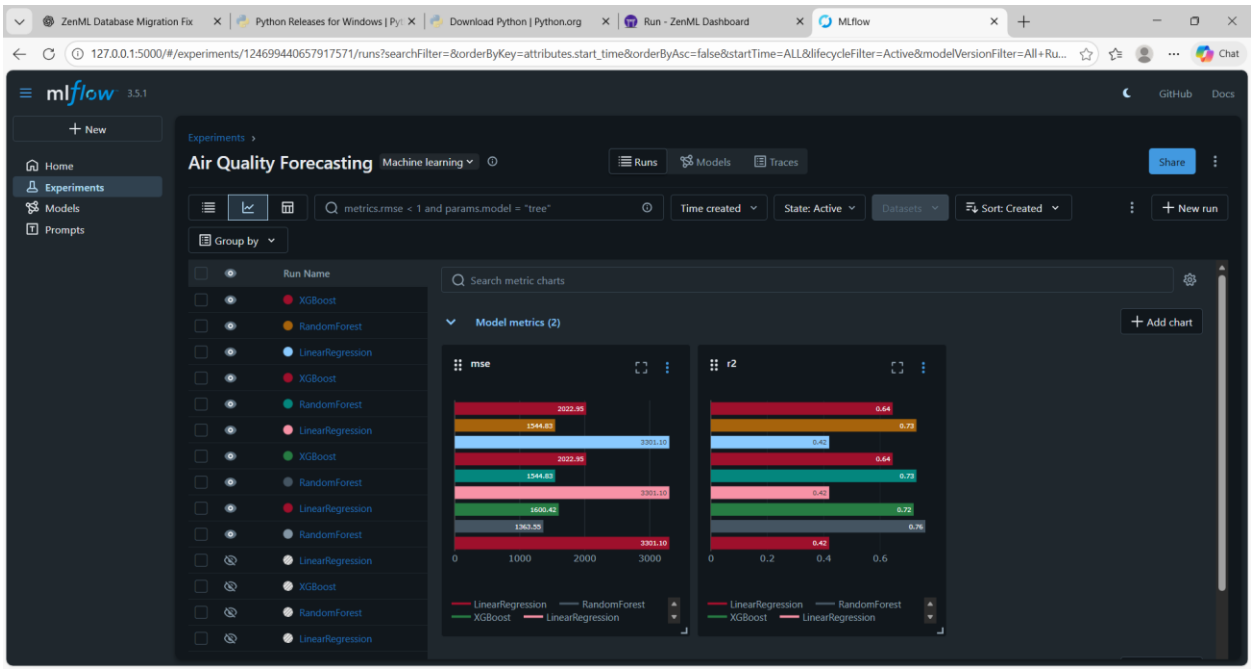


Figure 5.3: Model metrics graph from MLflow

5.3 Interpretability

Model interpretability is crucial for ensuring that AI-driven predictions are transparent, trustworthy, and actionable. In the context of air quality forecasting, it is important to understand which features, such as pollutant concentrations or meteorological factors, most significantly influence the predicted AQI values. This helps stakeholders, including environmental analysts and decision-makers, to trust the system and take informed actions based on the forecasts.

Feature-level analysis, such as **feature importance scores** from ensemble models, was conducted to identify the most influential variables. For example, fine particulate matter (PM2.5) and coarse particles (PM10) consistently emerged as the top contributors to AQI predictions, followed by weather parameters like temperature, humidity, and wind speed. Understanding these contributions helped in prioritizing data collection efforts and refining feature engineering steps.

Advanced interpretability techniques, including SHAP (SHapley Additive exPlanations) and partial dependence plots, were also used to visualize how changes in individual features affected model output. These techniques provided granular insight into model behavior, highlighted potential sources of bias, and guided improvements in model design, ensuring both robustness and explainability of the forecasts.

5.4 Model versioning and Registry

Model versioning and registry are critical components of a robust machine learning workflow, ensuring that every trained model can be tracked, reproduced, and deployed reliably. In the Air Quality Forecasting project, all models—including Linear Regression, Random Forest, and XGBoost—were **versioned** and registered to maintain clear records of their configurations, training datasets, and performance metrics.

The top-performing model was **registered as the production candidate**, while other trained models were retained as **archived versions**. This structure allows for quick rollback in case of performance drops and facilitates future retraining when updated datasets are available.

Implementing **model versioning** ensures that all models are fully **reproducible** and traceable, aligning with industry-standard **MLOps practices**. It provides a seamless transition from experimentation to deployment, supporting both reliability and maintainability of the air quality forecasting system.

5.5 Documentation of Model updates

Comprehensive **documentation** was maintained throughout the model development lifecycle to track all changes and decisions. This included:

- **Data preprocessing updates** (handling missing values, normalization, encoding)

- **Feature engineering steps** and rationale for feature selection
- **Algorithm selection decisions** and justification
- **Hyperparameter tuning experiments** and results
- **Evaluation metrics** for each model version

Maintaining such documentation ensures **transparency, reproducibility, and accountability**, making the system easier to maintain, scale, and enhance. It also facilitates **collaboration among team members** and supports auditing, troubleshooting, and future model improvements.

By linking documentation with **version control** and **MLflow experiment logs**, the project follows **end-to-end MLOps best practices**, ensuring a structured, traceable, and professional workflow from experimentation to deployment.

Chapter-6

Model Analysis and refinement

6.1 Overview of Pipeline Automation

In machine learning projects, models evolve not only due to code changes but also due to new data and feature updates. In our Air Quality Forecasting project, **ZenML pipelines** were used to orchestrate automated workflows for data preprocessing, model training, evaluation, and deployment. This ensures that the system remains **reproducible, consistent, and maintainable**, even as datasets or models are updated.

Key benefits of using ZenML pipelines:

- Automation of repetitive tasks (training, evaluation, preprocessing)
- Reproducibility of experiments and results
- Clear structure for orchestrating multiple models

CI/CD pipelines reduce manual intervention, minimize errors, and ensure consistent deployment of ML models while maintaining reproducibility and traceability.

6.2 Tools Setup and Integration

The following tools were integrated to build and analyze the models:

- **ZenML:** Orchestrates pipelines for preprocessing, feature engineering, training, and evaluation.
- **MLflow:** Tracks experiments, stores model artifacts, logs metrics, and manages versions.
- **Docker:** Packages trained models along with dependencies for reproducible deployment.
- **Evidently AI:** Monitors prediction distributions and detects data drift for post-deployment analysis.

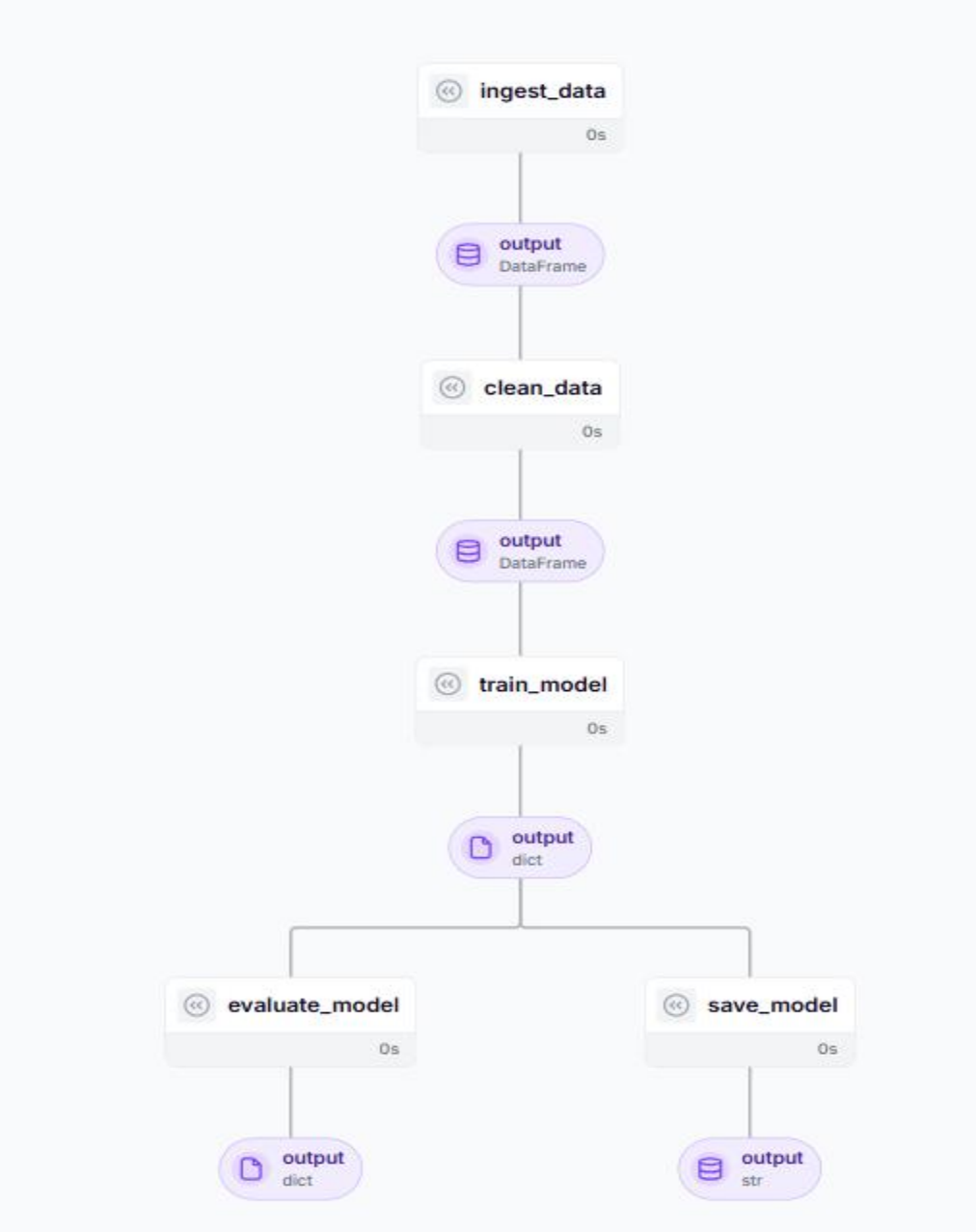


Figure 6.1: Zenml Pipeline

6.3 Automating Model Training and Testing

Whenever new air quality data or preprocessing updates were available, the ZenML pipeline executed the following steps:

1. Load and preprocess pollutant and meteorological data
2. Feature engineering (e.g., AQI scores, pollutant aggregates)
3. Train three models: **Linear Regression, Random Forest, and XGBoost**
4. Evaluate performance using **R^2, MAE, and MSE**

5. Log metrics, feature importance, and trained models in **MLflow**
6. Check model quality and select the best-performing candidate for deployment

The pipeline also ensured:

- Schema consistency
- Detection of missing or invalid values
- Overfitting checks via train–validation comparisons

6.4 Deployment Pipeline

Models passing evaluation were serialized and containerized using Docker. This guarantees:

- Environment consistency across local or cloud deployments
- Reproducibility, as each model version is tracked in MLflow
- Scalability, allowing smooth replacement when newer models outperform older versions

6.5 Monitoring and Model Refinement

Post-deployment, Evidently AI was used to monitor:

- Data drift and feature distribution changes
- Prediction stability under varying environmental conditions

Whenever drift or performance degradation was detected, the pipeline could be manually triggered to retrain models using the latest data. This ensures that the Air Quality Forecasting System remains accurate and reliable over time.

Chapter-7

Monitoring and tracking the model lifecycle

7.1. Model Monitoring Concepts and KPIs

Continuous **model monitoring** ensures that the Air Quality Forecasting system delivers **accurate and timely AQI predictions**, even as environmental patterns evolve. Unlike static software, ML models may degrade when pollution levels or meteorological conditions change. Two categories of monitoring metrics were defined:

7.1.1 Prediction Accuracy Metrics

- **R^2 Score:** Quantifies how much variance in actual AQI values is captured by the model.
- **Root Mean Squared Error (RMSE):** Penalizes larger prediction errors, helping avoid extreme mispredictions.
- **Prediction Distribution Checks:** Ensures forecasts are realistic and not biased toward extreme or constant values (e.g., always "Good" or "Hazardous").

7.1.2 System Performance Metrics

- Inference Latency: Measures the time required to generate AQI forecasts; high latency can delay alerts.
- Input Data Validation: Detects missing or malformed sensor readings (e.g., PM2.5, NO_2) before model inference.

7.2. Detecting Concept Drift and Data Drift

To maintain predictive reliability, **Evidently AI** was integrated into the inference pipeline for **drift detection**. It generates visual and JSON reports to compare incoming production data with the original training dataset. Two types of drift were monitored:

7.2.1. Data Drift (Covariate Shift)

Occurs when the statistical characteristics of inputs shift over time.

- **Example:** A sudden rise in particulate matter due to wildfires or seasonal pollution.
- **Detection:** K-S tests for numerical features (e.g., PM2.5, temperature) and Chi-Square tests for categorical features (e.g., city zones) identify significant deviations.

7.2.2. Concept Drift

Occurs when the link between input features and AQI changes.

- **Example:** Rainfall reduces particulate concentration, altering the previous correlation between PM2.5 and AQI.
- **Detection:** Comparison of predicted AQI with actual sensor readings highlights when model assumptions no longer hold.

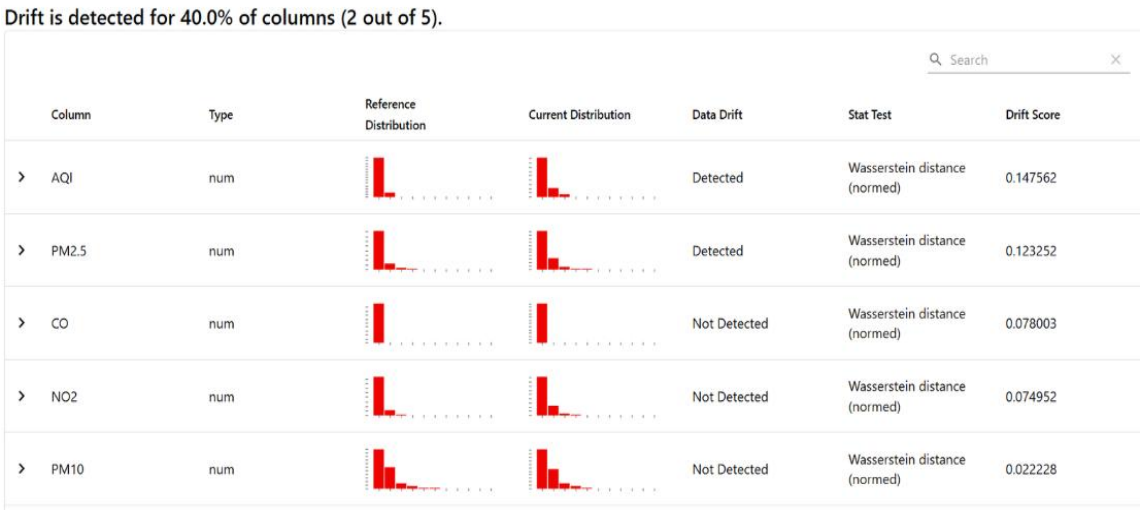


Figure 7.1: Dataset drift visualization using EvidentlyAI

7.3. Model Retraining and Redeployment

To ensure long-term reliability, a **Continuous Training (CT) pipeline** was established using **Prefect**:

- **Triggers:** Scheduled retraining (e.g., monthly) or conditional retraining when drift is detected (>30% of features).
- **Data Versioning:** DVC ensures the latest clean and validated sensor data is used for reproducible training.
- **Training Workflow:** Includes preprocessing, feature engineering, and fitting XGBoost or Random Forest models.
- **Evaluation:** New models are assessed on hold-out validation data.
- **Registration & Deployment:** High-performing models are registered in **MLflow Registry** (Staging), and CI/CD pipelines deploy the updated model in Docker containers to the live inference server.

7.4. Logging, Alerting, and Dashboard Setup

Effective monitoring requires **observability, alerts, and dashboards**:

- **MLflow Tracking:** Records hyperparameters, training metrics (MAE, RMSE, R^2), and artifacts (models, plots, preprocessing scripts).
- **Alerts:** Notifications via Slack/email for:
    - Drift exceeding 50% of monitored features
    - High inference latency (P95 > 500ms)
    - Prediction errors exceeding 5% in a short time window

- **Visualization Dashboard:** A **Streamlit interface** integrates Evidently AI reports, showing drift, AQI trends, and model performance to both technical and non-technical stakeholders.

Chapter-8

Performance Evaluation and Governance

8.1. Post-Deployment Performance Review

Once deployed, the Air Quality Forecasting system is continuously evaluated using **live or near-real-time environmental data** to verify that the model performs reliably under real-world conditions. The primary objective of this phase is to confirm that AQI predictions remain accurate, stable, and trustworthy after deployment.

Key evaluation metrics such as **Mean Absolute Error (MAE), R^2 score, prediction consistency, and alert accuracy** are recorded and monitored using **MLflow**. Performance trends across different model versions are analyzed to identify improvements or potential regressions over time. Based on this comparative analysis, the most reliable and well-performing model version is retained for production use.

This post-deployment evaluation framework ensures:

- **Accurate AQI forecasting**, enabling timely and reliable air quality alerts
- **Stable predictive behavior** despite seasonal variations and changing pollution patterns
- **Reduced false alarms and missed alerts**, improving public trust in the system

By routinely reviewing post-deployment performance metrics, the system prevents unnoticed performance degradation and ensures sustained forecasting quality and governance compliance.

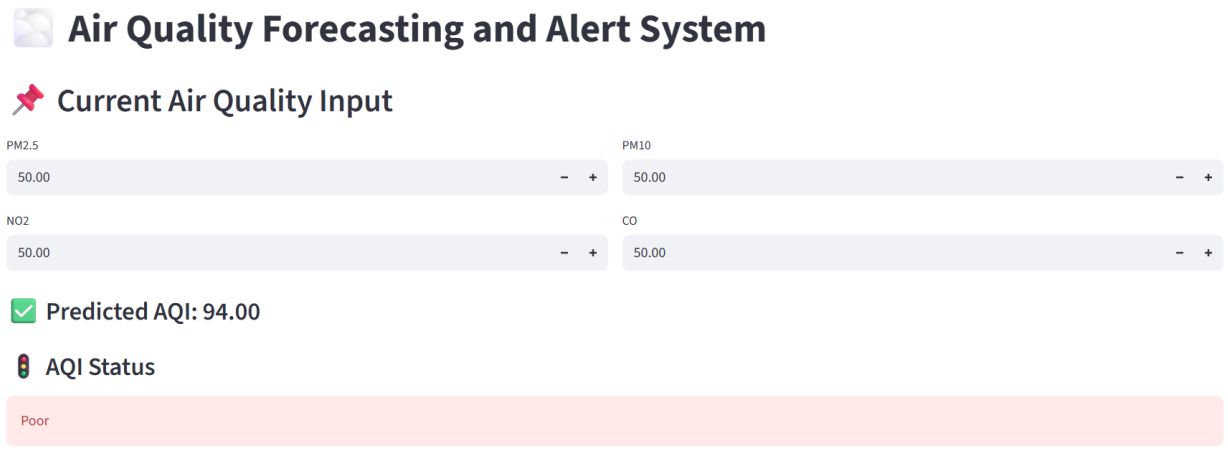


Figure 8.1: Input details and Model prediction

8.2. Continuous Improvement via Feedback Loops

To maintain long-term accuracy and relevance, the Air Quality Forecasting system employs **continuous feedback mechanisms**. After predictions are generated, actual air quality measurements (ground-truth AQI values) and alert outcomes are collected and stored. This real-world feedback is used to evaluate how closely the predicted AQI aligns with observed conditions.

The continuous improvement cycle involves:

- Augmenting datasets with newly collected pollutant and meteorological data
- Re-executing automated Prefect pipelines for preprocessing, feature engineering, model training, and evaluation
- Comparing updated models with previous versions using MLflow, ensuring that only models demonstrating measurable performance improvements are promoted to production

By following this iterative feedback-driven approach, the system gradually enhances forecasting accuracy, adapts to evolving environmental patterns, and ensures that AQI alerts remain reliable and timely for end users.

8.3. Governance and Ethical AI Practices

Strong governance mechanisms are essential to ensure that the Air Quality Forecasting and Alert System operates in a **responsible, transparent, and trustworthy** manner. Governance in this project addresses both **technical integrity** and **ethical use of predictive analytics**, particularly when forecasts influence public awareness and decision-making.

The key governance objectives include:

- **Accurate and unbiased AQI predictions**, ensuring that no specific region or time period is consistently misrepresented
- **Model transparency and explainability**, allowing stakeholders to understand which pollutants and environmental factors influence forecasts
- **Secure management of environmental and location-based data**, with controlled access and proper data handling practices
- **Responsible alert generation**, minimizing false alarms and missed warnings that could impact public trust

Ethical AI principles are upheld by regularly reviewing **feature importance and model behavior** to ensure that predictions are driven by scientifically relevant variables rather than spurious correlations. Periodic audits and performance reviews are conducted to verify compliance with

fairness, accountability, and responsible AI standards, ensuring that the system remains aligned with environmental governance and public safety objectives.

8.4. Documentation: Model Cards and Data Cards

To strengthen **transparency, accountability, and audit preparedness**, the Air Quality Forecasting and Alert System maintains structured documentation in the form of **Model Cards** and **Data Cards**. These artifacts provide a clear and standardized overview of both the predictive models and the datasets used throughout the project lifecycle.

Model Cards

Model Cards summarize critical information about each forecasting model, including:

- **Intended application** of the model, such as AQI prediction and early warning alerts
- **Machine learning techniques employed**, including Linear Regression, Random Forest, and XGBoost
- **Evaluation results** across training, validation, and production monitoring phases
- **Model constraints and risks**, such as sensitivity to rare pollution events or incomplete weather data

Data Cards

Data Cards describe the datasets used for training and evaluation, focusing on:

- Types of environmental data used, including air pollutants and meteorological attributes
- Data acquisition and preparation steps, covering cleaning, normalization, and transformation
- Derived features and temporal indicators used in AQI prediction
- Known data limitations, such as sensor outages or uneven geographic coverage

By maintaining these documentation artifacts, the project ensures **traceability, reproducibility, and interpretability**, enabling stakeholders to confidently understand and evaluate both the data and models powering the air quality forecasting system.

8.5. Audit and Review Checklist

A systematic audit and review process is adopted to ensure the reliability, transparency, and long-term sustainability of the Air Quality Forecasting and Alert System. Periodic audits help verify that the system continues to operate correctly, remains compliant with responsible AI practices, and delivers accurate air quality predictions.

The audit and review checklist includes:

- Verification of **MLflow experiment tracking**, including model versions, hyperparameters, and performance metrics
- Review of **Evidently AI reports** to monitor data drift, concept drift, and data quality issues
- Validation of **automated retraining workflows** implemented using Prefect, ensuring retraining is triggered appropriately when drift thresholds are exceeded
- Confirmation of **Docker-based deployment consistency**, ensuring the same model behavior across development, testing, and production environments
- Review of **project documentation**, including Model Cards and Data Cards, to ensure completeness, clarity, and audit readiness

Regular audits enable early identification of performance degradation, data issues, or deployment inconsistencies. This structured review process ensures that the system remains reliable, transparent, and ethically aligned while continuing to provide timely and accurate air quality forecasts and alerts.

Appendix

A.1 List Of MLOps Tools, Its Features, Pros And Cons

The project employs a set of MLOps tools to support model development, deployment, and monitoring. Each tool was used independently to address specific stages of the machine learning lifecycle. Full end-to-end tool integration was not implemented; instead, tools were applied in a modular manner based on project requirements.

1. MLflow

MLflow was used for tracking experiments and evaluating model performance during training.

Key Features

- Logging of training parameters and evaluation metrics
- Storage of trained model artifacts
- Comparison of multiple experimental runs

Pros

- Easy to use and lightweight
- Improves experiment traceability
- Helps in selecting the best-performing model

Cons

- Model registry was not fully utilized
- Does not provide production-level monitoring

2. ZenML

ZenML was used to structure the machine learning workflow and understand MLOps pipeline concepts.

Key Features

- Pipeline-based organization of ML steps
- Improves clarity of ML workflow stages
- Encourages reproducible pipeline design

Pros

- Helps standardize ML workflows
- Improves modularity of code
- Useful for learning MLOps best practices

Cons

- Not integrated with other MLOps tools in this project

- Advanced features were not utilized

3. Docker

Docker was used to containerize the application for consistent execution across environments.

Key Features

- Environment isolation
- Dependency management
- Container-based deployment

Pros

- Ensures consistency across development and deployment
- Simplifies application setup
- Reduces system compatibility issues

Cons

- Requires containerization knowledge
- Debugging containers can be complex

4. Evidently AI

Evidently AI was used to analyze data drift and model behavior using offline datasets.

Key Features

- Data drift detection
- Feature distribution comparison
- Automated HTML and JSON reports

Pros

- Provides clear visual insights
- Easy to apply to batch data
- Useful for post-deployment analysis

Cons

- Not real-time monitoring
- Computational overhead for large datasets

5. DVC (Data Version Control)

DVC was used to manage dataset versions alongside source code.

Key Features

- Dataset version tracking
- Reproducible data pipelines
- Integration with Git

Pros

- Maintains data consistency across experiments
- Enables reproducibility
- Scales well for large datasets

Cons

- Requires external storage configuration
- Initial setup effort

A.2 Tool Installation Guides

This section outlines the installation steps for the tools used in the Air Quality Prediction project. All tools were installed in a local development environment to support experimentation, version control, deployment, and monitoring.

1. MLflow

MLflow was installed to track experiments and evaluate model performance.

Installation Steps

- Create and activate a Python virtual environment
- Install MLflow using pip install mlflow
- Verify installation by running mlflow --version in the terminal

2. DVC (Data Version Control)

DVC was used to manage versions of air quality datasets and processed artifacts.

Installation Process

- DVC was installed using pip install dvc
- The project repository was initialized with dvc init
- Dataset versions were tracked alongside Git commits

3. ZenML

ZenML was used to define and manage machine learning pipelines such as data ingestion, preprocessing, model training, and evaluation.

Installation Process

- ZenML was installed using pip install zenml
- ZenML repository was initialized using zenml init
- Local ZenML stack was used for pipeline execution

4. Evidently AI

Evidently AI was used for offline monitoring of data drift and prediction behavior.

Installation Process

- Evidently AI was installed using pip install evidently
- Drift reports were generated using Python scripts

5. Docker

Docker was used to containerize the air quality prediction application for consistent deployment across environments.

Installation Process

- Docker Desktop (Windows/macOS) or Docker Engine (Linux) was installed
- Docker daemon was enabled
- Installation was verified using docker –version

A.3 References

1.5 Literature Review

[1] A. Sharma and P. Kumar proposed a fully integrated MLOps pipeline using **DVC** for dataset versioning and **MLflow** for experiment tracking. Their study demonstrates that linking specific Git commits to exact data snapshots and model metrics ensures full reproducibility, which is essential for reliable environmental forecasting systems.

[2] K. Patel and S. J. Raofocused on DVC as a cornerstone for continuous training pipelines, highlighting its storage-agnostic architecture for managing large-scale datasets. This approach is particularly relevant for air quality applications involving substantial pollutant and meteorological data.

[3] J. Nilsson et al. implemented **Evidently AI** for micro-batch monitoring, demonstrating that its drift detection algorithms can identify concept drift earlier than traditional batch monitoring. This capability supports timely recalibration of air quality predictors.

[4] R. Müller et al. evaluated Evidently AI against other drift detection tools on real-world environmental datasets. Their work found that Evidently's rich statistical tests and visualization reports make it highly effective for identifying shifts in feature distributions over time.

[5] Liu and Li studied the performance of statistical and machine learning models including Random Forest and LSTM for AQI prediction. Their results indicate that deep learning models can better capture non-linear pollutant dynamics than conventional regression.

[6] M. Rajesh et al. presented a machine learning framework for real-time air quality assessment and predictive risk mapping, showing that ML-based forecasting improves the early detection of high-pollution events.

[7] S. Zhao et al. developed an ensemble learning approach for AQI forecasting. Their study demonstrated that combined models reduce prediction error and increase robustness to fluctuating pollutant patterns.

[8] Zhang, Liu, and Chen investigated deep learning architectures such as CNN and RNN for multi-pollutant forecasting, concluding that hybrid models leveraging local spatial and temporal correlations perform significantly better.

[9] A. Kumar and R. Singh described an IoT-integrated air quality monitoring system, highlighting the importance of real-time data streams from sensors to support timely AQI forecasting and alerting.

[10] Nguyen, Tran, and Le introduced multi-source data fusion techniques using neural networks for pollutant forecasting. Their work emphasizes maintaining model accuracy in dynamic environmental conditions.

[11] Dhanavarshini et al. emphasized that combining statistical learning with environmental features enhances the understanding of pollutant behavior and improves AQI forecasting performance.

[12] Liu and Li compared several AQI forecasting techniques across different urban datasets. They found that models integrated with sensor networks and data analytics frameworks provide scalable and reliable predictions.

[13] Dhumale et al. evaluated classic ML algorithms for AQI prediction, showing that tree-based models like Random Forest and XGBoost balance interpretability with high forecast accuracy.

[14] Kaplan provided foundational insights into air quality data structures and forecasting methods, underscoring the necessity of rigorous preprocessing and feature engineering.

[15] Baklanov et al. explored integrated urban air quality forecasting frameworks, demonstrating that hybrid systems combining numerical models with empirical data outperform single-method approaches.

[16] Wang, Li, and Zhao analyzed ensemble and hybrid models for PM2.5 prediction, reporting that hybrid approaches reduce forecast errors and improve model stability.

[17] Park and Cho proposed spatio-temporal prediction using graph neural networks, showing that graph-based representations capture inter-station pollutant dependencies more effectively than traditional models.

[18] Roque et al. formalized automated alert mechanisms for environmental monitoring systems, proving that predictive modeling coupled with threshold-based alerts significantly improves early warning capabilities.

[19] Zaharia et al. introduced **MLflow**, detailing its capabilities for tracking the full machine learning lifecycle. Their work is widely cited for enabling reproducible experimentation in data-driven forecasting projects.

[20] Sharma and Gupta reviewed integration of ML pipelines with monitoring tools, highlighting that automated drift detection such as Evidently's is critical for maintaining prediction reliability over time.

[21] Idir et al. investigated data fusion from mobile and fixed air quality sensors, concluding that hybrid sensor networks improve spatio-temporal pollutant mapping and furnish richer datasets for AQI forecasting.

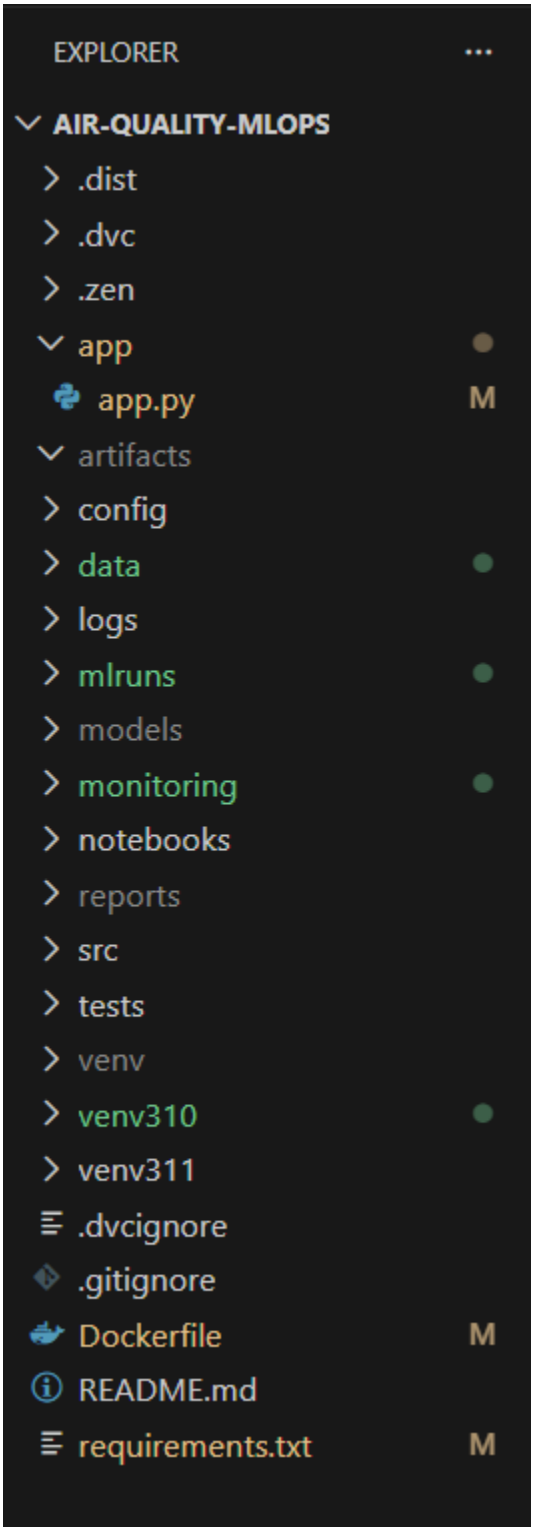
[22] Panja et al. developed a spatio-temporal graph convolutional model that uses extreme value modeling to better predict pollutant peaks, demonstrating improved forecast robustness.

[23] Bodendorfer proposed transformer-based air quality models that use attention mechanisms to improve prediction accuracy under complex meteorological conditions.

[24] Geng et al. presented a multimodal learning framework (FuXi-Air) integrating emission inventories, weather forecasts, and sensor data. Their method showed enhanced forecasting performance in large urban areas.

[25] Author and Mani provided a comprehensive review of hybrid air quality forecasting methods and highlighted that combining statistical and machine learning approaches yields more robust and accurate predictions.

A.4 Folder Structure



A.5 Further Reading

1. **DVC – https://dvc.org**
2. **MLflow – https://mlflow.org**
3. **Docker – https://docs.docker.com**
4. **Evidently AI – https://www.evidentlyai.com**

Dataset Link : Air Quality Data in India (2015 - 2020)